

Machine Vision Project 2

Serial Number Recognition

Antony Gerold Arockiasamy - 1404714
ENGG*6100 – Machine Vision
University of Guelph

February 24, 2026

Abstract

This report documents the development of a machine vision system for identifying automotive parts made of metal from their stamped serial numbers. The entire dataset contains 1,885 grayscale images across 12 part types, of which 1,872 images across 10 part types were selected for this project. Several approaches were tried and failed before arriving at a working solution. Direct thresholding and edge detection could not separate character signal from metal texture. A preprocessing pipeline using Gaussian blur, CLAHE, and black top-hat morphology successfully isolated characters, but character segmentation through connected components was unreliable at scale. This led to a holistic approach where the entire serial number region is classified as a single image using HOG features and KNN. With automatic ROI detection via MSER and KNN $k = 1$, the system achieves 96.3% part identification accuracy, confirmed by GroupKFold cross-validation at $96.4\% \pm 0.6\%$.

1 Introduction

The goal of this project is to build a system that can identify cast metal automotive parts by processing images of their stamped serial numbers. While the full dataset contains 1,885 grayscale images across 12 part types, this project uses a subset of 1,872 images across 10 part types. The dataset is not balanced, six parts have between 96 and 596 images, while four parts have only 13 or 14 images each. Two additional part types (20871905 and BK21-64842-A-B) containing 13 images total were excluded because they had too few images for reliable training and testing. Each part type has a unique serial number stamped into its surface.

At first glance this looks like a standard OCR problem. It is not. The casting process leaves a rough, bumpy texture on the metal surface, and the serial numbers are stamped (pressed) into this texture as shallow grooves. To the camera, the texture bumps and the character strokes have similar intensity and similar edges. Separating them turned out to be the main challenge of the entire project.

This report follows the order I actually worked through the problem. I start with what I tried first, explain why it failed, and describe how each failure pointed me toward the next approach. The final system uses automatic region detection, a morphological preprocessing pipeline, and holistic pattern matching to identify parts at 96.3% accuracy.

Table 1 shows the 10 part types used.

Table 1: Dataset summary.

Part Folder	Serial Number	Images
23354374	23354374	320
23301608	GM 23301608	416
23305982	GM 23305982	97
23477043	GM 23477043	596
23497667	GM 23497667	292
52611-0E110-A	52611-0E110-A	96
D0CW-51300	D0CW-51300	14
D0CW-51301	D0CW-51301	14
D0CW-51310	D0CW-51310	14
70085119-0300	70085119-0300	13
Total		1,872

2 Region of Interest

Each part type has its serial number in a different location on the image. Before any processing can happen, the serial number region needs to be found. I explored two approaches for this.

2.1 Manual ROI Calibration

The simplest method is to open one reference image per part, note the pixel coordinates of the serial number corners, and hard code them. I defined coordinates for all 10 parts this way. The downside is that it requires human setup for every new part type.

2.2 Automatic ROI Detection Using MSER

To reduce manual work, I tested automatic detection using MSER (Maximally Stable Extremal Regions) [6]. MSER finds regions in an image that stay stable across a range of intensity thresholds. Characters should show up as stable regions because they have clear boundaries against the metal background.

The approach works in three steps. First, MSER finds candidate regions on both the original and inverted image. Candidates are filtered by size and aspect ratio to keep only character-shaped blobs. Second, a density grid clusters these candidates spatially. The densest cluster is assumed to contain the serial number. Third, a bounding box is drawn around the cluster with some padding.

Testing across all 10 parts, the automatic ROI achieved an average recall of 90% and an average IoU of 0.13 against the manual ROI. For 9 out of 10 parts, the auto ROI successfully captured the serial number region, though with 3 to 10 times more background than needed. For one part (23477043), the MSER candidates clustered in the wrong area and missed the serial number entirely. Despite this, I proceeded with the auto ROI approach for all parts because 9 out of 10 is a reasonable detection rate for an unsupervised method. Figure 1 shows the comparison.

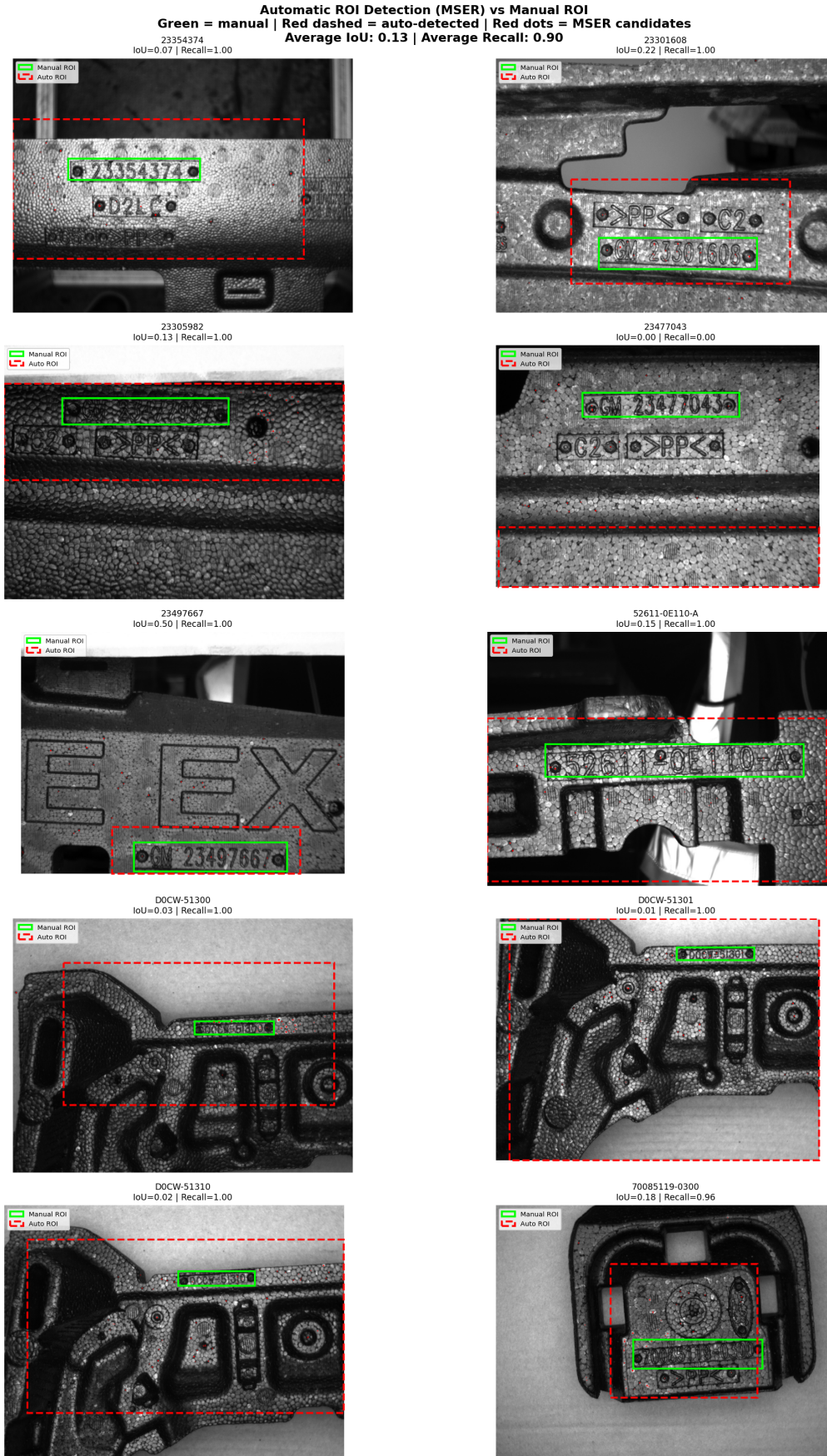


Figure 1: Automatic ROI detection (red dashed) vs manual ROI (green solid) across all 10 parts. The auto ROI captures the serial number but includes much more background.

I initially expected the wider auto ROI to hurt classification accuracy because it includes extra texture. As described in Section 5, the opposite turned out to be true. This was despite the fact that I initially tuned the preprocessing for the manual ROI to achieve better segmentation.

3 Preprocessing:

All processing is done on the cropped ROI region. I started with the simplest possible methods and built up from there.

3.1 Attempt 1: Direct Thresholding

The first thing I tried was converting the ROI to binary (black and white) using Otsu thresholding, adaptive thresholding, and a manual threshold. All three failed. The metal texture contains dark and light spots at the same intensity range as the character strokes. Thresholding treats a dark texture bump the same as a dark character pixel because it can only separate based on intensity. Figure 2 shows the results.

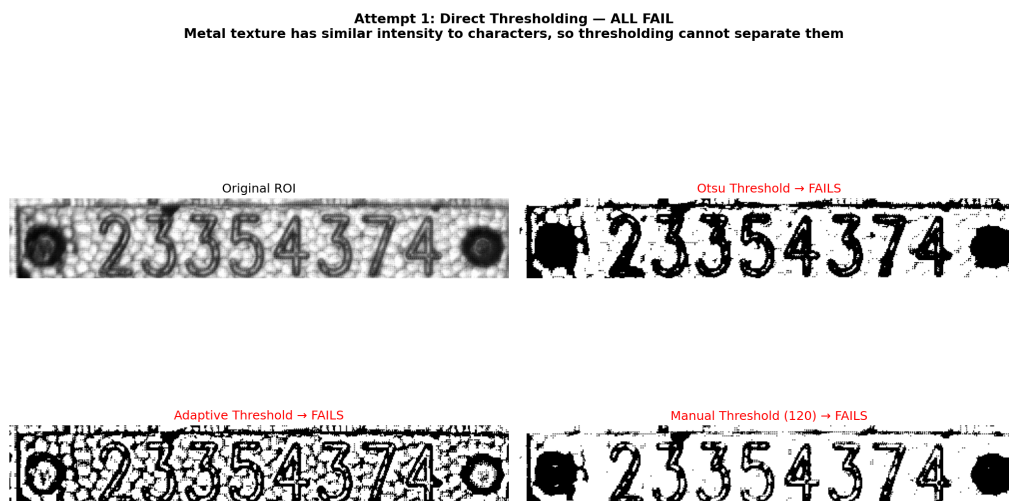


Figure 2: Direct thresholding fails on all methods. The texture noise has similar intensity to the characters.

3.2 Attempt 2: Edge Detection

If intensity cannot separate characters from texture, maybe edges can. Characters have strong edges at the boundaries of their strokes. I tested Canny and Sobel edge detection. Both failed because the textured metal surface is full of edges everywhere. Character edges are drowned out by texture edges. Figure 3 shows this.

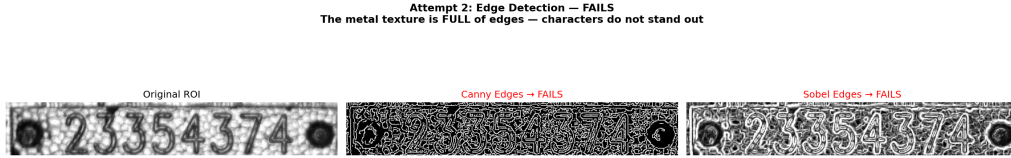


Figure 3: Edge detection fails. The metal texture produces edges everywhere, so characters do not stand out.

3.3 Attempt 3: Gaussian Blur Before Thresholding

The metal texture is high frequency. The characters are lower frequency. Gaussian blur smooths out high-frequency detail, so blurring before thresholding should suppress metal texture more than it suppresses characters.

I tested kernel sizes from 3 to 31. At small kernels (3 to 7), too much texture remains. At large kernels (23+), the characters themselves start degrading. There is a usable range around 11 to 15, but no single kernel perfectly separates texture from characters. Figure 4 shows the progression.



Figure 4: Gaussian blur at different kernel sizes. Small kernels leave texture; large kernels erode characters.

This told me blur helps but is not enough on its own. I needed a smarter extraction method.

3.4 Black Top-Hat Transform

The key insight came from thinking about how stamps physically work. A die presses down into soft metal, creating recessed valleys. These valleys reflect less light and appear darker than the surrounding surface. The black top-hat transform [1] is designed to extract exactly these kinds of dark features.

The black top-hat computes: $T_{black}(f) = (f \bullet b) - f$, where f is the image and b is the structuring element. In practice this works in two sub-steps. First, a morphological closing operation fills in the valleys by sliding a flat structuring element over the surface. This produces an image where the character recesses are gone. Second, subtracting the original from the closed result leaves only the filled-in parts, which are the characters.

I used a 25×25 rectangular structuring element. This size was chosen because character strokes are roughly 5 to 20 pixels wide, and the structuring element must be larger than the features being extracted. Before applying the top-hat, I added CLAHE (Contrast Limited Adaptive Histogram Equalization) [4] with $\text{clipLimit} = 3.0$ and $\text{tileGridSize} = 8 \times 8$ to normalize local contrast under uneven lighting.

Figure 5 shows the sub-steps and intensity profiles that confirm the transform isolates character locations.

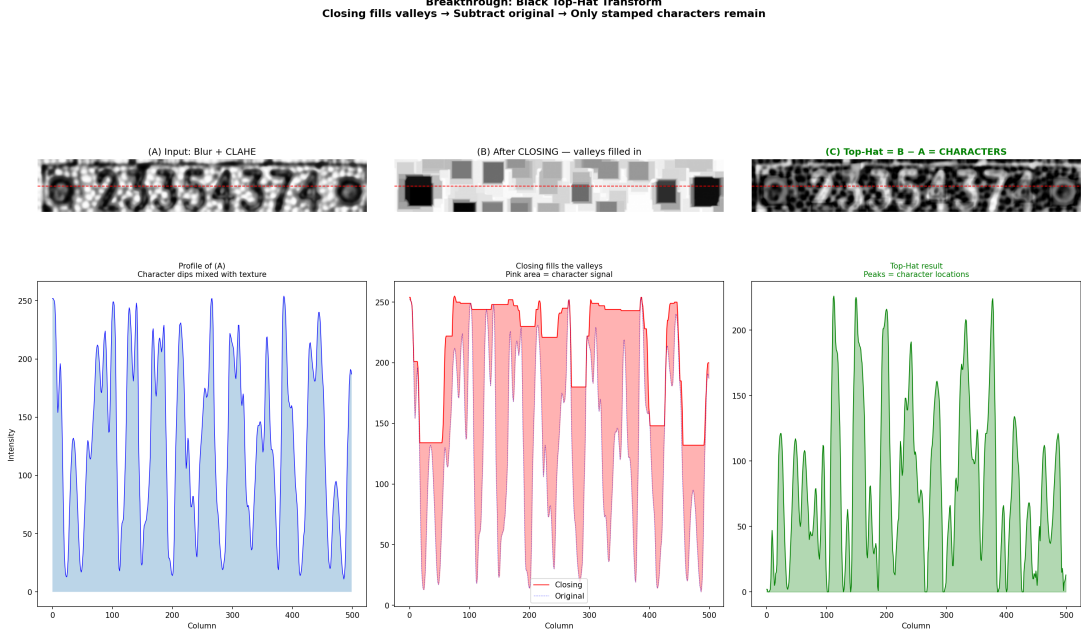


Figure 5: Black top-hat transform. (A) Input after blur and CLAHE. (B) After closing, valleys are filled. (C) Top-hat result: only the character valleys remain. The intensity profiles below confirm peaks correspond to character positions.

3.5 Complete Preprocessing Pipeline

The final preprocessing pipeline applies five operations in sequence: (1) Gaussian blur with a per-part kernel size (9 to 25), (2) CLAHE, (3) black top-hat with a 25×25 kernel, (4) Otsu thresholding [5] to convert to binary, and (5) morphological opening with a 3×3 kernel to remove small noise specks. Figure 6 shows each stage.

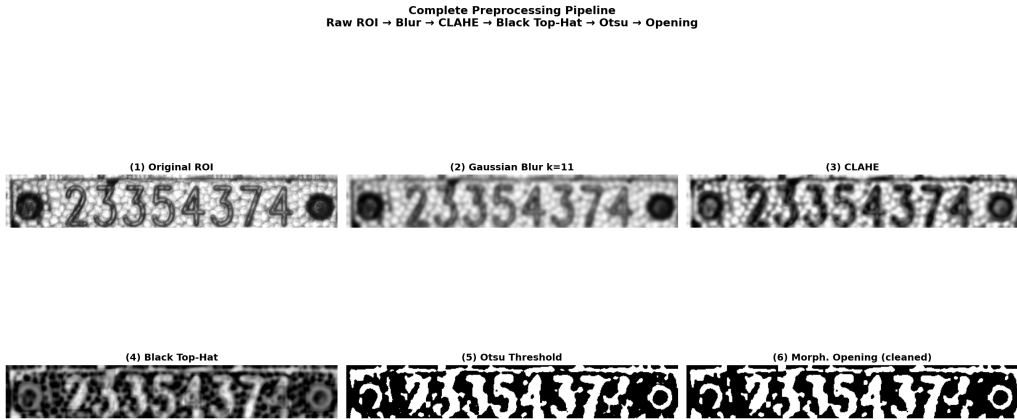


Figure 6: Complete preprocessing pipeline on part 23354374.

It is worth noting that the Otsu binarization step causes some information loss. The grayscale top-hat output preserves the intensity difference between strong character edges and weak texture edges. Once binarized, this distinction is gone. This becomes important later when choosing between character segmentation and holistic classification.

4 Character Segmentation:

With the preprocessing working, I tried to isolate individual characters using connected component analysis on the binary output. Components were filtered by height ($>12\%$ of ROI height), width ($<18\%$ of ROI width), area (>80 pixels), and aspect ratio (<2.0).

4.1 Results on Reference Images

Testing on one reference image per part, only 2 out of 10 parts produced blob counts within ± 2 of the expected character count. The remaining 8 parts produced too many blobs because residual texture noise creates false components. Figure 7 shows the binary output for all 10 parts.

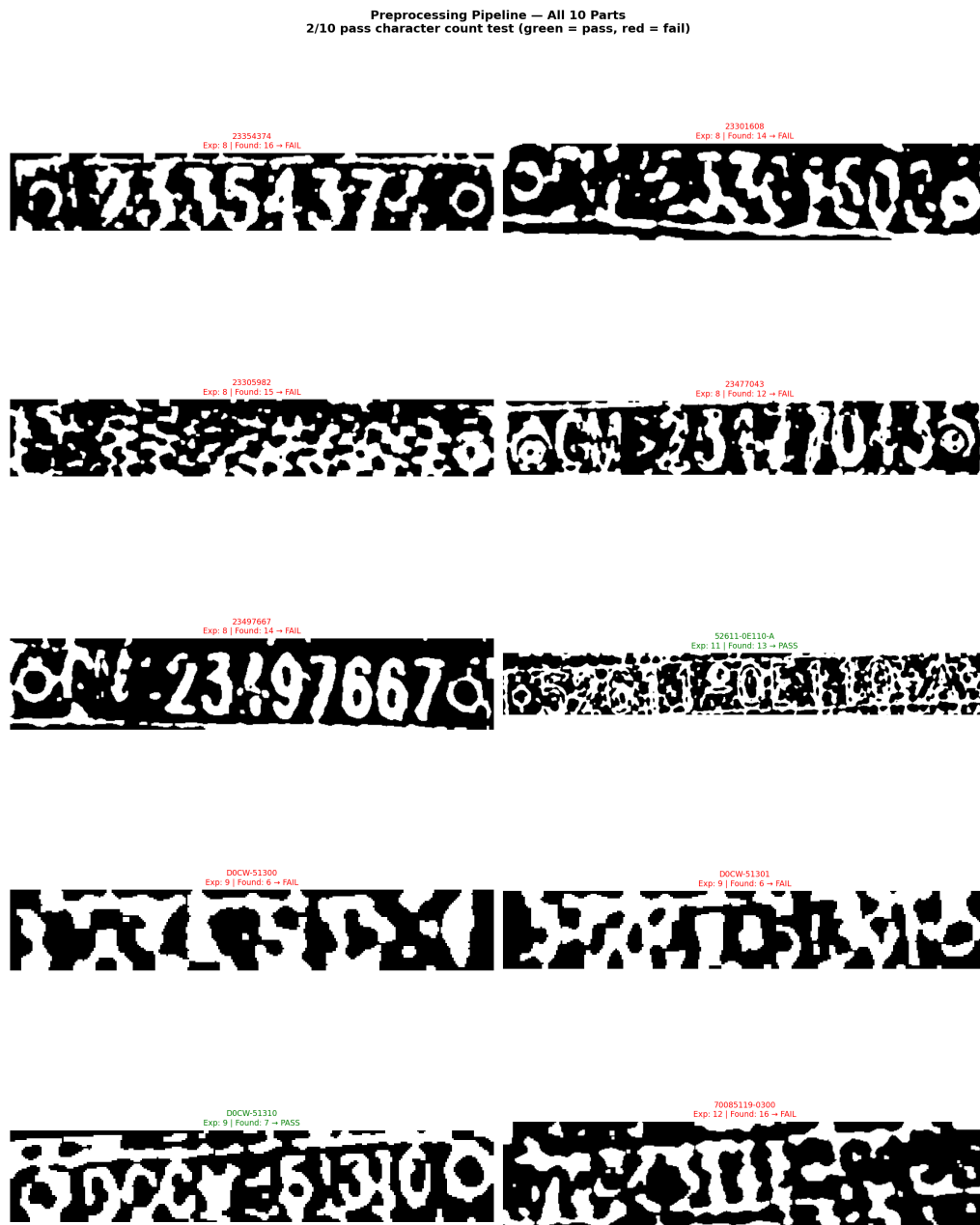


Figure 7: Preprocessing pipeline applied to all 10 parts. Only 2/10 produce correct blob counts. The rest have too many false blobs from texture noise.

4.2 Character Extraction and Classification

From the parts where segmentation worked, I extracted 11 labeled character crops (28×28 pixels) by matching blobs left-to-right against the known serial numbers. Figure 8 shows these crops. Most do not look like recognizable characters because the texture noise is processed into the binary output.

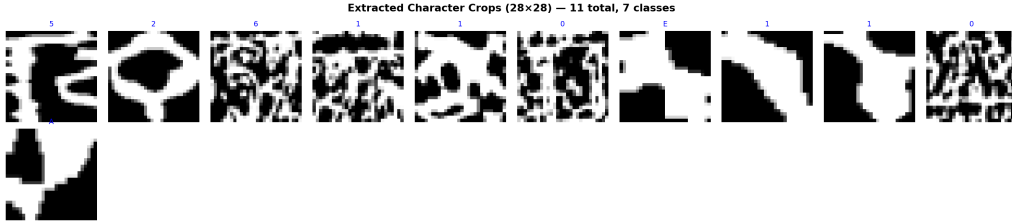


Figure 8: Extracted character crops. Most are corrupted by texture noise.

Even with data augmentation (rotations, translations, erosion/dilation), a KNN classifier trained on these samples cannot work at scale because the input quality is too poor. The classifier is not the problem. Segmentation is the bottleneck. It cannot reliably produce clean character crops from the binary output.

4.3 Conclusion: Segmentation Does Not Scale

The core issue is that binarization destroys the intensity information that distinguishes characters from texture. In the grayscale top-hat output, characters are bright and texture is dim. After Otsu thresholding, both become white pixels. Connected component analysis then finds hundreds of blobs with no way to tell which are characters and which are noise.

This failure led me to rethink the entire approach. Instead of trying to segment individual characters, I would classify the serial number region as a whole.

5 Holistic Classification

5.1 The Idea

Each of the 10 part types has a unique serial number. That means each part produces a unique visual pattern in its serial number region, even if individual characters cannot be cleanly isolated. Instead of segmenting characters, I treat the entire preprocessed ROI as a single image and extract a feature vector that serves like a fingerprint for that part type.

5.2 Feature Extraction with HOG

HOG (Histogram of Oriented Gradients) [2] captures the distribution of edge directions in local regions of an image. It is commonly used for shape based recognition.

The pipeline for each image is crop the ROI, apply blur, CLAHE, and black top-hat. Its the same preprocessing pipeline as before but stopping at the grayscale top-hat output, without thresholding, resize to 128×64 pixels, and extract HOG features with

9 orientations, 8×8 pixel cells, and 2×2 cell blocks. This produces a 3,780-dimensional feature vector per image.

The important difference from the segmentation approach is that HOG operates on the grayscale top-hat output, not the binary image. This preserves the intensity distinction between characters (strong edges) and texture (weak edges), which binarization would destroy.

5.3 Classification with KNN

KNN (K-Nearest Neighbors) [3] classifies each test image by finding the most similar training image in feature space. The entire dataset of 1,872 images is split 80/20 into 1,497 training and 375 test images.

5.4 Manual ROI vs Automatic ROI

I tested both ROI approaches with the same HOG + KNN pipeline to see which performs better.

Table 2: Manual ROI vs Automatic ROI with HOG + KNN $k = 1$.

ROI Method	Accuracy	Notes
Manual ROI (tight)	89.3%	Only serial number region
Auto ROI (MSER, wide)	96.3%	Serial number + surrounding context

The automatic ROI outperformed manual by 7 percentage points. The wider detection area captures additional visual context around the serial number (screw holes, casting edges, part geometry) that differs between part types. HOG picks up these features and KNN uses them for classification. While this means the system is partly identifying parts by their surrounding structure rather than purely the serial number, this additional information makes the classifier more robust, especially for the D0CW parts whose serial numbers differ by only a single digit.

6 Results

6.1 Final System Performance

Table 3 shows KNN accuracy at different k values using the automatic ROI approach.

Table 3: Classification accuracy at different KNN k values.

Configuration	Accuracy
KNN $k = 1$	96.3%
KNN $k = 3$	93.9%
KNN $k = 5$	92.8%
GroupKFold CV ($k = 1$)	96.4% $\pm$ 0.6%

KNN $k = 1$ works best because each image of a given part is most similar to other images of that same part. Higher k values risk including neighbors from visually similar but different parts. The GroupKFold cross-validation result ($96.4\% \pm 0.6\%$) confirms the accuracy is stable and not dependent on a lucky train/test split.

6.2 Per-Class Performance

Table 4: Per-class results for Auto ROI + HOG + KNN $k = 1$.

Part	Images	Precision	Recall	F1
23354374	320	0.97	1.00	0.98
23301608	416	0.99	0.99	0.99
23305982	97	1.00	0.95	0.97
23477043	596	0.92	1.00	0.96
23497667	292	1.00	0.90	0.95
52611-0E110-A	96	1.00	0.89	0.94
D0CW-51300	14	1.00	0.67	0.80
D0CW-51301	14	1.00	0.67	0.80
D0CW-51310	14	1.00	1.00	1.00
70085119-0300	13	1.00	0.33	0.50

Parts with 96 or more images all achieve F1 scores above 0.94. The weaker results come from parts with very few images (13 to 14 each). With only 3 test images per small class, a single misclassification drops accuracy by 33 percentage points. Precision is 1.00 for all small classes, meaning when the system does make a prediction for these parts it is always correct. It just sometimes fails to recognize them (lower recall).

6.3 Approach Comparison Summary

Table 5 lists every approach tried in the order I attempted them.

Table 5: All approaches attempted in chronological order.

Approach	Method	Result	Key Issue
Direct thresholding	Otsu, adaptive	FAIL	Texture = same intensity
Edge detection	Canny, Sobel	FAIL	Texture full of edges
Blur + threshold	Gaussian + Otsu	Partial	No perfect kernel size
Full pipeline	Blur+CLAHE+TopHat	Works	Characters isolated
Character segmentation	Conn. components	2/10	Too many false blobs
Holistic, manual ROI	HOG + KNN $k = 1$	89.3%	Works but tight ROI limits context
Holistic, auto ROI	MSER + HOG + KNN	96.3%	Extra context helps

7 Discussion

The most important design decision was using the black top-hat transform, which was motivated by the physics of how stamps create recessed valleys in metal. The 25×25

structuring element size follows directly from the character stroke width (5 to 20 pixels). Gaussian blur was chosen over median blur because Gaussian provides smoother texture suppression without preserving sharp texture edges.

The key finding of this project is that character segmentation is the bottleneck for cast metal serial number recognition with classical methods. The classifier works well when given clean input, but the binarization step required for segmentation destroys the intensity information needed to separate characters from texture.

The holistic approach avoids this bottleneck entirely by running HOG on the grayscale top-hat output. It sacrifices the ability to read individual characters, but for the task of identifying parts from a fixed set, this tradeoff is worthwhile.

The auto ROI result was unexpected. I initially assumed tighter crops would give better features, but the additional context captured by the wider MSER detection actually helps. This is especially true for the D0CW parts, whose serial numbers differ by one digit and are nearly indistinguishable from the serial number alone.

To verify that the 23477043 failure was not inflating the overall accuracy, I tested the system with that part excluded. Accuracy on the remaining 9 parts was 97.3%, which is higher than the 96.3% with all parts included. This confirms that 23477043 was the weakest performer, not artificially boosting the score. The system still classifies 23477043 correctly most of the time because the auto ROI captures a consistent region across all 596 images of that part, giving KNN a repeatable pattern even though it is based on part geometry rather than the serial number.

The main limitation is dataset imbalance. The four parts with fewer than 15 images account for most errors. Collecting even 50 additional images per underrepresented part would likely improve their accuracy significantly. Another limitation is that the system cannot identify part types it has not been trained on.

8 Conclusion

I developed a part identification system through iterative experimentation. Direct thresholding and edge detection failed because the metal texture has similar intensity and edge characteristics to the stamped characters. A preprocessing pipeline using Gaussian blur, CLAHE, and black top-hat morphology successfully isolated characters in grayscale, but binarization for character segmentation introduced too much noise for reliable connected component analysis.

The final system uses MSER based automatic ROI detection, the same morphological preprocessing (stopping before binarization), HOG feature extraction on the entire region, and KNN $k = 1$ classification. It achieves 96.3% accuracy on 1,872 images across 10 part types, confirmed by GroupKFold cross-validation at $96.4\% \pm 0.6\%$.

9 Tools and Citations

All code was written in Python using OpenCV [7] for image processing, scikit-image [8] for HOG feature extraction, and scikit-learn [9] for KNN classification and evaluation. Matplotlib was used for all figures.

References

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [2] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in *IEEE CVPR*, 2005, pp. 886–893.
- [3] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Trans. Inform. Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [4] S. M. Pizer et al., “Adaptive histogram equalization and its variations,” *Comput. Vision Graph. Image Process.*, vol. 39, no. 3, pp. 355–368, 1987.
- [5] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Trans. Syst. Man Cybern.*, vol. 9, no. 1, pp. 62–66, 1979.
- [6] J. Matas et al., “Robust wide-baseline stereo from maximally stable extremal regions,” *Image and Vision Computing*, vol. 22, no. 10, pp. 761–767, 2004.
- [7] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [8] S. van der Walt et al., “scikit-image: image processing in Python,” *PeerJ*, vol. 2, e453, 2014.
- [9] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011.